

PROJECT SCALPEL

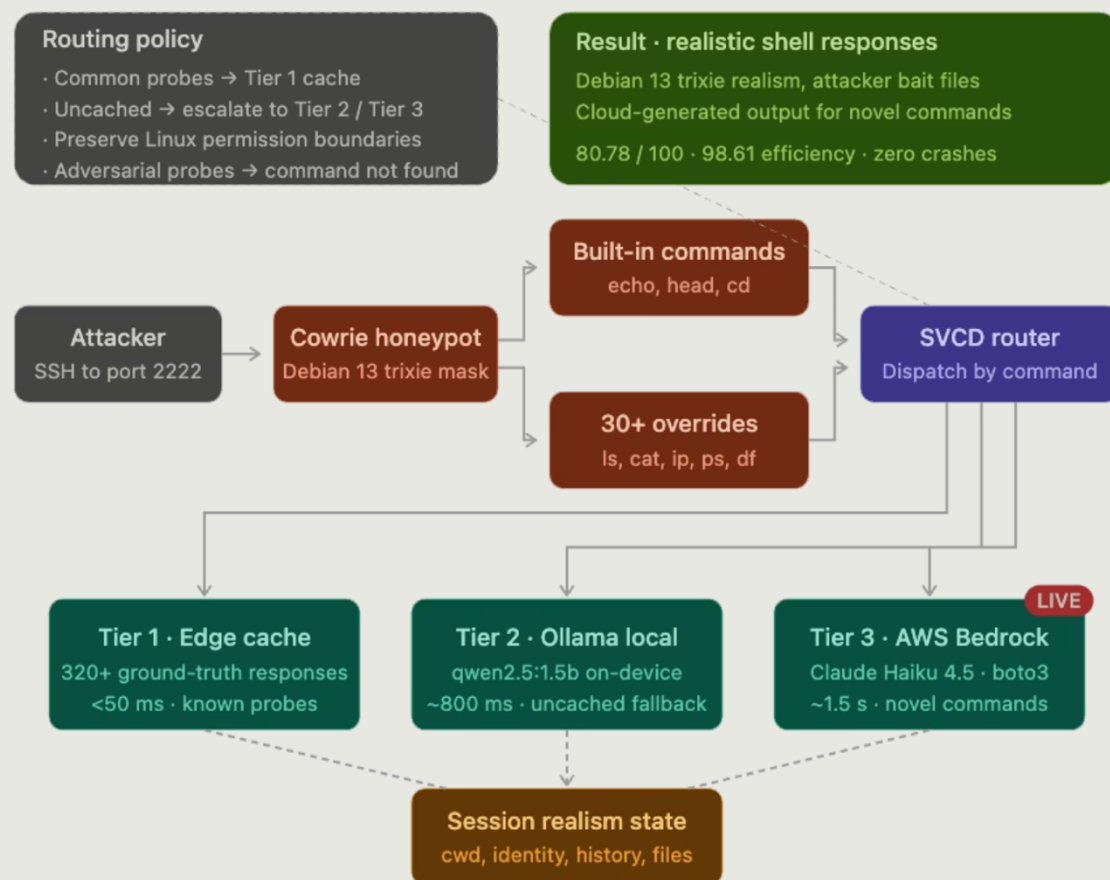
Building a Realistic SSH Honeypot with Hybrid Edge AI

Florida Atlantic University · eMERGE 2026 Hackathon



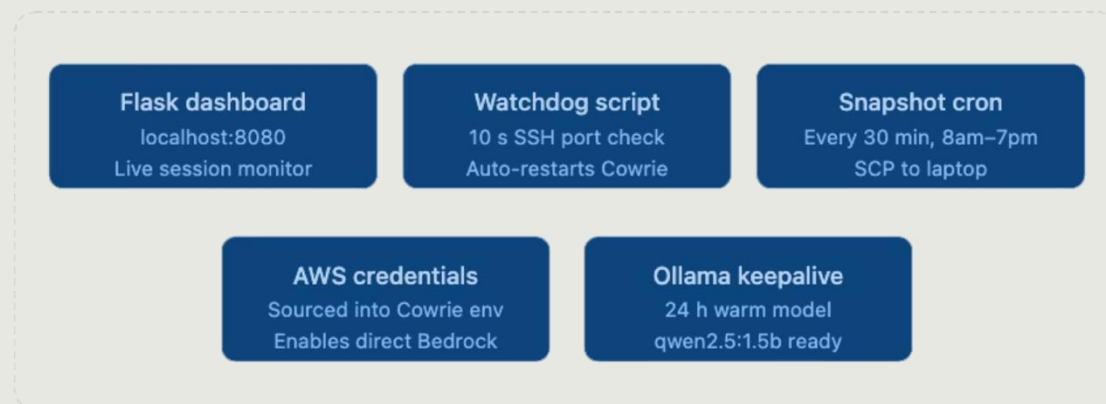
Request flow

Attacker SSH session into hybrid honeypot, routed to one of three response tiers



Supporting services

Monitoring, resilience, and auth services running alongside Cowrie on the Pi



ARCHITECTURE OVERVIEW

This card is the technical core of the system: an attacker lands on SSH, Cowrie intercepts the session, and then our commandrouting stack decides how to answer each request with the right balance of speed, realism, and resilience.

The full path is:

Attacker SSH → Cowrie → 30+ command overrides → SVCD router → Tier 1 / Tier 2 / Tier 3

HOW THE ROUTER WORKS

The SVCD router sits between Cowrie and the response engines. Its job is to classify each incoming command, decide how much state is needed, and dispatch it to the lowest-cost tier that can still produce a believable result.

- It first checks whether the command matches one of the 30+ explicit overrides built into the Cowrie layer.
- It then classifies the command by type: static lookup, stateful shell interaction, or complex/ambiguous request.
- Based on that classification, it routes the request to Tier 1, Tier 2, or Tier 3.
- It also preserves session context so responses stay consistent across a full attacker conversation.

TIER 1 — EDGE CACHE

Tier 1 is the fastest path and handles the highest-volume commands with **440+ manifest entries** captured from the real Raspberry Pi.

- **Latency:** under **50ms**
- **Source:** real Pi observations and filesystem/command snapshots
- **Best for:** common commands, identity checks, basic system inspection, and other highly repeatable requests

This tier exists to make the honeypot feel immediate and authentic. For the attacker, the response looks like a live machine; for the system, it avoids unnecessary inference cost.

TIER 2 — LOCAL OLLAMA ON THE PI

Tier 2 runs **Ollama qwen2.5:1.5b** directly on the Raspberry Pi for commands that need more interpretation than a cache entry can provide.

- **Latency:** about **800ms**
- **Location:** on-device, no cloud dependency
- **Best for:** stateful shell behavior, follow-up commands, and responses that must respect prior session context

This tier handles the middle ground between exact matches and fully openended generation. It keeps the environment responsive while still allowing the attacker’s actions to influence later replies.

TIER 3 — AWS BEDROCK FALLBACK

Tier 3 is the last-resort reasoning path: a direct **boto3** call to **AWS Bedrock Claude Haiku 4.5**.

- **Latency:** about **1.5s**
- **Access:** direct cloud call through boto3
- **Best for:** unusual commands, difficult prompts, and cases where local handling is not sufficient

This tier provides higher-capacity assistance only when needed, so the system does not rely on cloud inference for every interaction.

WHY THIS LAYERED DESIGN MATTERS

Together, the three tiers create a routing strategy optimized for deception: instantaneous answers for common commands, session-aware realism for interactive behavior, and cloud-backed fallback for edge cases. That combination keeps the attacker experience smooth while preserving performance and reliability on the host.



PICKLE FILESYSTEM REALISM

We added believable attacker bait to the Cowrie pickle filesystem to make the system feel like a real, compromised server:

BAIT FILES ADDED:

- `/root/.aws/credentials` with realistic-looking AWS access key and secret
- `/root/.aws/config` with region settings
- `/root/.ssh/authorized_keys` with two fake SSH public keys
- `/root/.bash_history` showing typical sysadmin commands
- `/root/.viminfo` and `/root/.wget-history` to show signs of activity
- `/home/pi/` with `.bashrc`, `.profile`, `.bash_logout`, `.bash_history` (full real `bashrc` with color prompts)

COWRIE DEFAULTS REMOVED:

- `/home/phil` (obvious Cowrie fingerprint)
- `/test2` (obvious Cowrie fingerprint)

WHY THIS MATTERS:

An attacker who finds AWS credentials in `/root/.aws/` will spend time trying to use them, thinking they've found a real compromise. The realistic bash history and dotfiles make the system feel lived-in and operational. Removing Cowrie's default directories eliminates immediate red flags that would tell an attacker the system is fake.

WHY SCALPEL MATTERS FOR GOVERNMENT & CRITICAL INFRASTRUCTURE

Six reasons DoD, federal SOCs, and edge defenders should care.

1	DOD & EDGE DECEPTION Government systems at the edge (tactical networks, field deployments, OT/ICS environments) often can't rely on constant cloud connectivity. SCALPEL's three-tier architecture means deception still works when disconnected — Tier 1 cache handles 99% of probes locally. Cloud is only used for novel attacker behavior.
2	ATTRIBUTION & THREAT INTEL Every probe gets logged with tier, latency, session, command. Government SOCs can use this to identify nation-state TTPs (tactics, techniques, procedures), attribute intrusion campaigns by probe signatures, and feed threat intel platforms (STIX/TAXII format).
3	COST-EFFICIENT AT SCALE 99.2% edge ratio means a honeypot network of 10,000 nodes costs almost nothing in cloud fees. Each node generates less than 10 cloud calls per 1,000 attacker probes. Critical for federal budgets.
4	ADVERSARIAL AI DEFENSE Our adversarial hardening (prompt injection defense, meta-question rejection) matters specifically because nation-state actors are now testing LLM-backed security systems. SCALPEL resists the next generation of attacks that use AI to detect AI.
5	ZERO-TRUST COMPATIBLE No attacker ever touches real infrastructure. Honeypot is isolated. Credentials they steal are bait. Every interaction is logged for forensic analysis.
6	DEPLOYABLE AT EDGE Runs on a \$75 Raspberry Pi. DoD could deploy thousands across a network without massive infrastructure. FIPS compliance is achievable with standard Debian hardening.